

Esame OS Internals (Cabodi)

Data: 16/01/2026

Nota Generale: In tutte le domande, ogni volta che il risultato è una selezione, un SI/NO, o un numero, sono necessari una motivazione/spiegazione o passaggi intermedi che portino al risultato.

SECTION1: THEORETICAL PART

Domanda 1 (Gestione Memoria e Paginazione)

Contesto: Un sistema di calcolo ha le seguenti caratteristiche:

- Dimensione della memoria fisica: 32 GB.
- Spazio di indirizzamento virtuale: indirizzato con 48 bit, indirizzamento al byte.
- Dimensione della pagina: 8 KB.
- Tabelle delle pagine organizzate in modo gerarchico, assumendo che ogni tabella nella gerarchia occupi esattamente una pagina e che ogni Page Table Entry (PTE) abbia dimensione 8 byte.
- TLB con 256 entry.
- Tempo di accesso alla memoria principale (RAM): 100 ns.
- Tempo di lookup della TLB: trascurabile rispetto alla memoria RAM.

A) Calcolare:

1. Il numero di bit dell'offset di pagina.
2. Il numero di bit del numero di pagina virtuale (Virtual Page Number).
3. Quante Page Table Entry sono contenute in una singola tabella delle pagine.
4. Quanti bit del virtual page number possono essere tradotti (al massimo) da una tabella delle pagine a due livelli.
5. Quanti bit del virtual page number possono essere tradotti (al massimo) da una tabella delle pagine a tre livelli.
6. Il numero minimo di livelli della tabella delle pagine necessario per mappare l'intero spazio di indirizzamento virtuale di un processo.

B) Calcoli sui tempi di accesso:

Assumendo che la tabella delle pagine abbia il numero minimo di livelli calcolato al punto A.6. Ricordando che in caso di TLB hit, la traduzione richiede solo il lookup in TLB e un accesso in memoria; in caso di TLB miss, la traduzione richiede un accesso in memoria per ogni livello della tabella delle pagine, più un accesso in memoria per il dato.

1. Calcolare il tempo di accesso in caso di TLB miss (in ns).
2. Determinare il minimo hit ratio della TLB tale che il tempo medio di accesso (EAT) non superi 120 ns.

Domanda 2 (File System e Allocazione Indicizzata)

Contesto: Un file system utilizza un'allocazione indicizzata con due livelli di blocchi indice (indice gerarchico a 2 livelli). Non si tratta di una struttura Unix/UFS basata su inode: la struttura è esattamente la seguente:

- Un blocco indice di primo livello che contiene puntatori a blocchi indice di secondo livello.
- Ogni blocco indice di secondo livello contiene puntatori ai blocchi dati.

I parametri del sistema sono: dimensione blocco su disco = 4 KB; dimensione di un puntatore = 32 bit; partizione = 2 TB (gli altri metadati possono essere trascurati).

A) Calcolare la massima dimensione di un file supportata da questo schema di allocazione. Fornire il risultato in GB.

B) Un file binario ha dimensione 73.011 KB (dove $1 \text{ KB} = 2^{10} \text{ byte}$). Calcolare:

1. Numero di blocchi dati allocati.
2. Numero di blocchi indice di secondo livello allocati.
3. Numero di blocchi indice di primo livello allocati.
4. Frammentazione interna nei blocchi dati (in byte).
5. Frammentazione interna nei blocchi indice (in byte).

C) Si assuma che il file in (B) contenga numeri reali in doppia precisione (64 bit ciascuno). Calcolare:

- Il numero del blocco dati logico (l'indirizzo logico del blocco) che contiene l'elemento con indice $n = 250.000$ (indice degli elementi 0-based).
- Quale blocco indice di secondo livello e quale entry all'interno di esso devono essere acceduti per raggiungere quel blocco dati.

D) Per ciascuna affermazione indicare SI/NO (Vero/Falso):

- Aumentare la dimensione del blocco su disco (mantenendo fissa la dimensione del puntatore) aumenta la dimensione massima del file.
- Con questo schema di allocazione, l'accesso casuale (random access) a un blocco può essere effettuato con un numero $O(1)$ di letture di blocchi indice (cioè un numero costante di blocchi indice).
- Il numero di blocchi indice di secondo livello utilizzati da un file cresce approssimativamente in modo lineare con la dimensione del file.
- I blocchi indice soffrono necessariamente di frammentazione interna anche quando la dimensione del file è un multiplo esatto della dimensione del blocco.

Domanda 3 (Memoria Virtuale e I/O)

Contesto: Si consideri un sistema con memoria virtuale a paginazione a richiesta, MMU con TLB, I/O su file tramite SO, dispositivi I/O con DMA, supporto per memory-mapped files e page replacement gestito dal kernel. Per ciascuna affermazione indicare SI/NO:

- **A)** Usando file mappati in memoria, l'accesso a un byte del file può causare un page fault anche se non viene eseguita alcuna system call di I/O in modo esplicito.
- **B)** Durante un trasferimento DMA verso un buffer utente, le corrispondenti pagine virtuali devono rimanere residenti in memoria fisica per tutta la durata del trasferimento.
- **C)** Se una pagina coinvolta in un trasferimento DMA in corso venisse spostata su disco (swap-out), potrebbe verificarsi corruzione dei dati.
- **D)** La presenza di una TLB garantisce che non siano necessari ulteriori accessi in memoria per tradurre indirizzi virtuali durante operazioni di I/O a blocchi.
- **E)** Aumentare la dimensione della pagina può ridurre il numero di page fault causati da accessi sequenziali a file, ma può aumentare il costo di accessi casuali (random) al file.
- **F)** Quando un file viene letto sequenzialmente usando `read()`, il kernel può sovrapporre (overlap) I/O e calcolo più facilmente rispetto al caso in cui il file sia acceduto tramite memory mapping.
- **G)** Un page fault causato dall'accesso a un file mappato in memoria può bloccare il processo chiamante anche se l'I/O su disco necessario a soddisfare il fault viene effettuato tramite DMA.

1 SECTION 2: OS161

Domanda 4 (Configurazione OS161)

Contesto: È dato un sistema OS161. Si supponga di aver aggiunto le seguenti istruzioni a `kern/conf/conf.kern`:

```
defoption project
optfile project syscall/project.c
```

e di aver creato il file `PROJECT` in `kern/conf`, con l'aggiunta della riga `options project`. Dire se le affermazioni seguenti sono vere (SI) o false (NO):

- **A)** L'esecuzione di `./config PROJECT`, seguita dal comando `bmake depend`, è sufficiente affinché il file opzionale `syscall/project.c` sia compilato quando si esegue `bmake`.
- **B)** Il file `project.h` viene generato automaticamente dal comando `./config PROJECT`.
- **C)** Il file `opt-project.h` viene generato solo se l'istruzione `options project` compare nel file `PROJECT`.
- **D)** Il file `opt-project.h` contiene la definizione `#define opt_project`.
- **E)** Si supponga di inserire in `main.c` l'istruzione `project_init();`. Per fare in modo che l'istruzione sia compilata solo se l'opzione è abilitata, andranno condizionate le istruzioni come segue:

```
#ifdef OPT_PROJECT
    project_init();
#endif
```

oppure come segue:

```
#if OPT_PROJECT
    project_init();
#endif
```

Domanda 5 (Syscalls e Spinlock OS161)

A) Funzione `sys_getpid`:

Si consideri la funzione `pid_t sys_getpid(void);`. Indicare SI/NO:

1. La funzione è necessaria per implementare `sys_waitpid`, perché è necessario il pid del processo in attesa.
2. La funzione è necessaria per implementare la chiamata di sistema `getpid` e verrà chiamata direttamente da un processo utente.
3. La funzione è necessaria per implementare la chiamata di sistema `getpid`, e sarà chiamata dal kernel, nella funzione `syscall`.
4. La funzione restituisce il pid del processo corrente.
5. La funzione viene chiamata per ottenere il pid di un altro processo (quello di cui attendere la terminazione).

6. La funzione viene chiamata da `sys_exit`, in modo da conoscere il pid del processo a cui inviare una segnalazione.

B) Funzioni Spinlock:

Si considerino le funzioni OS161 `spinlock_data_get` e `spinlock_data_testandset`. Indicare SI/NO:

1. Per implementarle è necessario utilizzare dei semafori (al fine di risolvere problemi di mutua esclusione).
2. Le funzioni al loro interno chiamano `spinlock_acquire` e `spinlock_release`.
3. Le funzioni sono utilizzate per realizzare `spinlock_acquire`.
4. La funzione `spinlock_release` non chiama la `spinlock_data_testandset` ma la `spinlock_set`, perché non deve gestire una mutua esclusione.